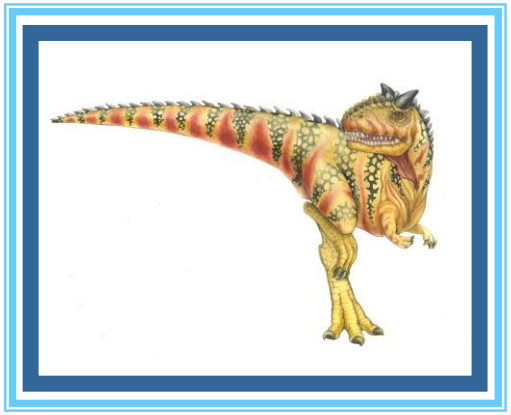
Chapter 5: Synchronization Tools

فصل پنجم: ابزار همگام سازی



Silberschatz, Galvin and Gagne ©2013

سرفصل

- پیش زمینه
- مشکل بخش بحرانی
- راه حل پترسون
- پشتیبانی سخت افزار برای همگام سازی
- قفل های درگیری متقابل(Mutex locks)
- سمافورها(Semaphores)
- مانیتورها(Monitors)
- سرزندگی(Liveness)
- ارزیابی



اهداف

- توصیف مسئله بخش بحرانی و توضیح یک وضعیت رقابتی (Race Condition)
- توضیح راه‌حل‌های سخت‌افزاری برای مسئله بخش بحرانی با استفاده از مانع‌های حافظه (memory barriers)، عملگرهای مقایسه و تعویض (compare-and-swap) و متغیرهای اتمی
- نشان دادن اینکه چگونه قفل‌های متقابل، سمافورها، مانیتورها و متغیرهای شرطی می‌توانند مسئله بخش بحرانی را حل کنند



اهداف

در این بخش موارد زیر شرح داده می شود:

- مشکل بخش بحرانی و نمایش یک وضعیت مسابقه (Race Condition)
- انواع راه حل های برای مشکل بخش بحرانی
- چگونگی استفاده از قفل های انحصار متقابل، سمافورها، مانیتورها و
- متغیرهای شرطی برای حل مشکل بخش بحرانی



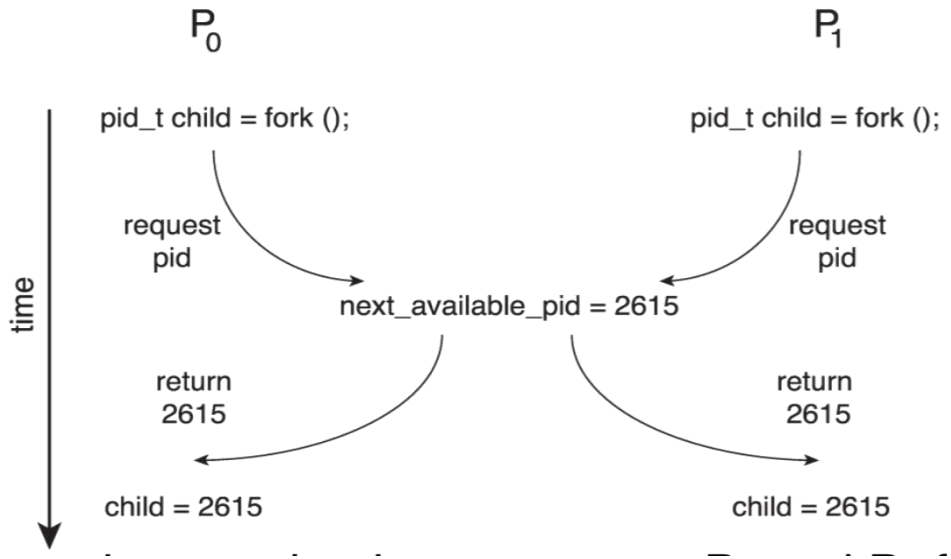
یادآوری

- فرآیندها می‌توانند هم‌زمان اجرا شوند
- ممکن است در هر زمان قطع شوند و اجرای آنها به صورت ناقص ادامه یابد
- دسترسی هم‌زمان به داده‌های مشترک ممکن است به ناهماهنگی داده‌ها منجر شود
- حفظ سازگاری داده نیازمند مکانیزم‌هایی است که اجرای منظم فرآیندهای همکار را تضمین کنند
- در شرایط مختلفی ممکن است یک متغیر به صورت هم‌زمان به‌روزرسانی می‌شد باعث ایجاد وضعیت رقابتی (Race Condition) شود.



شرایط رقابتی Race Condition

- فرایندهای `P0` و `P1` در حال ایجاد فرایندهای فرزند با استفاده از فراخوانی سیستم `fork()` هستند
- یک وضعیت رقابتی روی متغیر کرنل `next_available_pid` وجود دارد
- این متغیر نشان‌دهنده آی‌دی بعدی فرایند (`pid`) است.
- اگر مکانیزمی وجود نداشته باشد که از دسترسی هم‌زمان `P0` و `P1` به این متغیر جلوگیری کند،
- ممکن است یک `pid` یکسان به دو فرآیند مختلف اختصاص یابد!





مسئله بخش بحرانی

- سیستمی شامل nفرآیند را در نظر بگیرید {p0, p1, … pn-1}
- هر فرآیند بخشی از کد را دارد که بخش بحرانی نامیده می‌شود، فرآیند ممکن است متغیرهای مشترک را تغییر دهد، جدول‌ها را به‌روزرسانی کند، فایل بنویسد و …
- وقتی یک فرآیند در بخش بحرانی است، هیچ فرآیند دیگری نباید وارد بخش بحرانی خود شود.
  - مسئله بخش بحرانی این است که پروتکلی طراحی کنیم که این قانون رعایت شود
- هر فرآیند باید برای ورود به بخش بحرانی، در قسمت ورود (entry section) اجازه بگیرد،
- پس از بخش بحرانی وارد بخش خروج (exit section) شود، و سپس به بخش باقی‌مانده (remainder section) برود



ساختار عمومی فرآیند Pi با ناحیه بحرانی

```
while (true) {
        entry section        ناحیه ورودی
            critical section  ناحیه بحرانی
        exit section         ناحیه خروجی
            remainder section
}
```



انواع محاوره فرا یندها

- فرایندهای مستقل: (بی اطلاعی فرایند ها از یکدیگر) اینها فرایند های مستقل از یکدیگرند خواستار همکاری با یکدیگر نیستند و تبادل اطلاعاتی ندارند.

- فرایندهای همکار: دو روش برای تبادل داده بین فرایندهای همکار وجود دارد.

  - اطلاع غیر مستقیم فرایند ها از یکدیگر(استفاده از حافظه مشترک): اینها فرایند هایی هستند که لزوما از شناسه یکدیگر آشنا نیستند، ولی در دسترسی به بعضی اشیاء مثل بافر ورودی خروجی با یکدیگر مشترکند.

  - اطلاع مستقیم از یکدیگر (ارسال و دریافت پیام): اینها فرایندهایی هستند که قادرند با استفاده از شناسه، با یکدیگر ارتباط برقرار کنند و برای کار مشترک بر روی بعضی فعالیت ها ساخته شده اند. دستورات `send` و `receive` بجای حافظه مشترک استفاده می شود.



مباحث مطرح در ارتباط بین فرایندها

در طراحی سیستم عامل، سه موضوع زیر در رابطه با ارتباط بین فرایندها مطرح است:

- همگام سازی (Synchronization)
  - اگر بین فرایندها وابستگی وجود داشته باشد، ترتیب درست انجام کارها باید رعایت شود.
- تبادل اطلاعات (Communication)
  - فرایندها می توانند با مکانیسم هایی چون "حافظه مشترک، تبادل پیام، فایل مشترک و لوله" با یکدیگر تبادل اطلاعات کنند.
- رقابت فرایندها
  - فرایندها درفعالیت های بحرانی یکدیگر مداخله نکنند و شرایط رقابتی (Condition Race) برای آنها رخ ندهد.



انواع محاوره فرایندها

میزان اطلاع	رابطه	تاثیری که یک فرایند روی فرایند دیگری میگذارد	مسئله بالقوه کنترل
بی اطلاعی فرایند ها از یکدیگر	رقابت	• استقلال نتایج یک فرایند از عملکرد فرایندهای دیگر • امکان تاثیر در زمانگیری فرایند	• انحصار متقابل • بن بست • گرسنگی
اطلاع غیر مستقیم فرایند ها از یکدیگر	همکاری بوسیله اشتراک	• امکان وابستگی نتایج یک فرایند به اطلاعات بدست آمده از فرایند های دیگر • امکان تاثیر در زمانگیری فرایند	• انحصار متقابل • بن بست • گرسنگی • وابستگی داده ها
اطلاع مستقیم از یکدیگر	همکاری توسط ارتباط	• امکان وابستگی نتایج یک فرایند به اطلاعات بدست آمده از فرایند های دیگر • امکان تاثیر در زمانگیری فرایند	• بن بست • گرسنگی



رقابت میان فرایند ها برای منابع

در مورد فرایند های رقیب با سه مسئله کنترلی مواجهیم:

- انحصار متقابل(بخش بحرانی)
  - در هر لحظه فقط یک برنامه اجازه دارد به بخش بحرانی خود وارد شود.
  - به عنوان مثال در هر لحظه تنها یک فرایند اجازه دارد پیامی را به چاپگر بفرستد.

- بن بست
  - هنگام اعمال انحصار متقابل، در صورتیکه یک فرایند کنترل منبعی را در اختیار بگیرد و در انتظار منبع دیگری برای اجرا یاشد ممکن است بن بست رخ دهد.

- گرسنگی(قحطی)
  - ممکن است یکی از فرایندهای مجموعه برای مدتی نامحدود از دسترسی به منابع مورد نیازش محروم بماند، چرا که سایر فرایند ها منابع را به طور انحصاری بین یکدیگر مبادله میکنند. به این حالت گرسنگی می گویند.



انحصار متقابل(بخش بحرانی)

- فرض کنید چند فرایند برای دسترسی به یک منبع غیر اشتراکی مانند چاپگر رقابت می کنند. در طی اجراء، هر یک از فرایندها، فرمان هایی را به دستگاه ورودی/ خروجی ارسال می کنند. چنین منبعی را منبع بحرانی و بخشی از برنامه که از آن استفاده می کند را بخش بحرانی آن برنامه می گوییم.

- مهم این است که در یک زمان، تنها یک برنامه مجاز است تا در بخش بحرانی خود باشد.

- بخش هایی از برنامه که رفتار آنها با عوامل مشترک، ایجاد رقابت می کنند، را ناحیه بحرانی (Critical Region) می گویند.



بن بست (Deadlock)

- اعمال انحصار متقابل دو مسئله کنترلی، بن بست وگرسنگی را به وجود می آورد.
- دو فرایند P1 و P2 و دو منبع بحرانی R1 و R2 مفروض است که هر یک از فرایندها برای انجام عمل خود به هر دو منبع نیاز دارند. اگر منبع R1 به P2 و منبع R2 به P1 داده شود، هر یک منتظر منبع دیگر می باشند و هیچ یک از فرایندها، منبعی را که در اختیار دارد را رها نمی کند تا فرایند دیگر آن را دریافت کرده و بخش بحرانی خود را انجام دهد. بنابراین هر دو فرایند در بن بست قرار می گیرند.
- به زبان ساده بن بست یعنی تا ابد منتظر ورود به ناحیه بحرانی خود بودن.



گرسنگی، قحطی

- فرض کنید هر یک از سه فرایند P1 و P2 و P3 متناوباً نیازمند دسترسی به منبع R هستند.
- وقتی P1 این منبع را در اختیار گیرد، P2 و P3 در انتظار آن منبع، به تاخیر انداخته می شوند. با خروج P1 از ناحیه بحرانی، یکی از P2 یا P3 باید R را در اختیار گیرند. اگرP3 منبع R را بگیرد و قبل از پایان بخش بحرانی مجدداً P1درخواست R را بکند و بعد از پایان P3 ، اجازه به P1 داده شود و مکرراً این عمل بین P1 و P3 ادامه یابد، P2 به صورت نامحدود از دسترسی به منبع R محروم می ماند و گرسنگی می کشد.
- به عبارت دیگر گرسنگی یعنی به مدت نامعلوم و بدون حد بالای مشخص، منتظر فرایندهای دیگر بودن.



رویکردهای انحصار متقابل برای بخش بحرانی

راه حل های مشکل ناحیه بحرانی ملزوماتی دارد، به عبارت دیگر برای اینکه همکاری درست و کارا بین فرایندهای هم روند برقرار باشد، باید شروطی داشته باشد از جمله:

- انحصار متقابل (Mutual Exclusion)
  - از بین فرایندهایی که برای یک منبع یکسان دارای ناحیه بحرانی هستند، در هر لحظه فقط یک فرایند مجاز است که در ناحیه بحرانی خود باشد.
- پیشرفت (Progress)
  - فرایندی که فعلا تصمیم به ورود به ناحیه بحرانی را ندارد و در ناحیه غیربحرانی می باشدو دستورالعمل های عادی برنامه خود را اجرا می کند، نباید در تصمیم گیری برای ورود فرایندهای دیگر به ناحیه بحرانی شرکت کند.(امکان ممانعت نداشته باشد)
- انتظار محدود (Bounded Waiting)
  - باید مدت انتظار فرایندهایی که نیاز به ورود به ناحیه بحرانی دارند، محدود باشد. یعنی نباید دچار گرسنگی و بن بست شوند.
- البته علاوه بر رعایت سه شرط فوق، مسئله را باید در حالت کلی حل کرد و هیچ فرضی درباره سرعت نسبی فرآیندها وجود ندارد. فرض می‌شود هر فرآیند با سرعت غیر صفر اجرا می‌شود.



راه حل های مسئله انحصار متقابل

برای تحقق انحصار متقابل، پیشنهادهای مختلفی وجود دارد. این راه حل ها را به صورت چهار رویکرد دسته بندی می کنیم:

1) با حمایت سخت افزار(با کمک دستورالعمل های خاص CPU)

2) نرم افزاری

3) با حمایت سیستم عامل (با کمک فراخوان های سیستمی خاص)

4) با حمایت زبان برنامه سازی (با کمک کامپایلر)



رویکردهای انحصار متقابل با حمایت سخت افزار

می توان به کمک راه حل هایی که از دستورالعمل های ویژه ماشین استفاده می کنند، و نیاز به حمایت ازطرف پردازنده دارند، مسئله انحصار متقابل را حل کرد. این راه حل ها عبارتند از:

دستورالعمل از کار انداختن وقفه ها:

- در این راه حل، هر فرایند باید به محض ورود به ناحیه بحرانی، تمام وقفه ها را از کار بیندازد و دقیقا قبل ازخروج از ناحیه بحرانی، همه وقفه ها را مجددا فعال سازد. در این صورت پردازنده نمی تواند از فرایندی به فرایند دیگر سوئیچ کند، چون وقفه ساعت غیر فعال است. بنابراین وقتی که فرایندی وقفه ها را غیر فعال میکند، می تواند بدون ترس از دخالت فرایندهای دیگر، به خواندن و نوشتن در حافظه مشترک بپردازد.



راه‌حل مبتنی بر وقفه‌ها

- بخش ورود (Entry section): غیرفعال کردن وقفه‌ها
- بخش خروج (Exit section): فعال کردن دوباره وقفه‌ها

- آیا این راه‌حل مشکل را حل می‌کند؟
- اگر بخش بحرانی یک قطعه کد باشد که یک ساعت اجرا طول می‌کشد چه؟
- آیا ممکن است بعضی فرآیندها گرسنگی بکشند (هرگز وارد بخش بحرانی نشوند)؟
- اگر سیستم دو CPU داشته باشد چه اتفاقی می‌افتد؟



رویکردهای انحصار متقابل با حمایت سخت افزار

دستور العمل TSL(Test and Set Lock)

- بسیاری از کامپیوترها دارای دستورالعمل TSL می باشند. این دستور محتویات یک کلمه از حافظه (lock)را خوانده و در ثبات قرار می دهد. سپس مقدار 1 را در همان آدرس از حافظه ذخیره می کند. این عملیات غیر قابل تقسیم انجام می شوند و تا اینکه اجرای دستورالعمل تمام نشود، هیچ فرایند و یا پردازنده دیگری نمی تواند به این کلمه از حافظه دسترسی پیدا کند.



رویکردهای انحصار متقابل با حمایت سخت افزار

دستورالعمل SWAP

- می توان از دستورالعملی به نام swap، برای نوشتن یک روال ورود به ناحیه بحرانی استفاده کرد. این دستور می تواند در یک عمل واحد غیر قابل تقسیم، محتویات یک رجیستر پردازنده را ها محتویات یک کلمه حافظه، جابجا کند.



رویکردهای نرم افزاری انحصار متقابل

- راه حل های نرم افزاری مستقیما توسط برنامه ها استفاده می شوند و وجود حافظه اشتراکی ضروری می باشد.
- در این راه حل ها از دستورالعمل های خاص توسط سخت افزار استفاده نمی‌شود و حمایتی ازسیستم عامل و زبان های برنامه سازی نداریم.
- روشهای ارائه شده عبارتند از:
  - الگوریتم Decker (پنج مرحله تلاش)
    - تناوب قطعی ، استفاده از متغیر سراسری مشترک turn
    - استفاده از پرچم مشترک flag
    - استفاده از متغیرهای قفل lock
    - استفاده از متفیر نوبت turn و پرچم flag بصورت همزمان
  - الگوریتم پترسون



راه کار نرم افزاری ۱

- راه حل دو فرآیند
- فرض کنید دستورالعمل های بارگذاری و ذخیره سازی زبان ماشین اتمیک هستند؛ یعنی قابل وقفه نیستند .دو فرآیند یک متغیر مشترک دارند:
- `int turn;`
- تغییر `turn` نشان می دهد که نوبت کدام فرآیند است تا وارد بخش بحرانی شود .
- مقدار اولیه `turn` برابر با `i` تنظیم می شود.



Algorithm for Process Pi

```
while (true){

        while (turn = = j);

        /* critical section */

        turn = j;

        /* remainder section */

}
```



راه کار نرم افزاری ۱ برای دو فرایند

- مقدار اولیه turn برابر با یک بوده و p1 به ناحیه بحرانی می رود درحالی پردازش دیگر در حلقه وایل منتظر هست. پس از خروج پردازش p1 متغیر turn برابر 0 می شود. حال p0 وارد ناحیه بحرانی شده و خارج می شود. مقدار turn یک شده و پردازش p1 وارد ناحیه بحرانی می شود و در حین خروج turn =0 می شود. اگر فرض کنیم پردازش p0 در ناحیه باقی مانده به صورت طوالنی باشد. شرط پیشرفت برای p1 نقض خواهد شد. زیرا p1 پس از تغییر turn=0 دیگر نمی تواند وارد ناحیه بحرانی خود شود.

```
Process P0:
while (true){

    while (turn = = 1);

    /* critical section */

    turn = 1;

    /* remainder section */

}
```

```
Process P1:
while (true){

    while (turn = = 0);

    /* critical section */

    turn = 0;

    /* remainder section */

}
```



راه حل بهبود یافته پترسون(راه کار نرم افزاری)

- فرض کنید دستورالعمل های بارگذاری و ذخیره سازی زبان ماشین اتمی هستند؛ یعنی قابل وقفه نیستند .دو فرآیند دو متغیر مشترک دارند:

  - `int turn;`
  - `boolean flag[2]`

- متغیر `turn` نشان می دهد که نوبت کدام فرآیند است تا وارد بخش بحرانی شود .آرایه `flag` برای نشان دادن آمادگی یک فرآیند برای ورود به بخش بحرانی استفاده می شود مقدار فلگ برابر `true` میشود و نشان می دهد که فرآیند `Pi` آماده است.

```
flag[i] = true
```



راه حل بهبود یافته پترسون (راه کار نرم افزاری)

```
while (true){

        flag[i] = true;
        turn = j;
        while (flag[j] && turn = = j)
                ;

            /* critical section */

        flag[i] = false;

        /* remainder section */

}
```



راه حل بهبود یافته پترسون (راه کار نرم افزاری)

```
Process P0:
while (true){

        flag[0] = true;
        turn = 1;
        while (flag[1] && turn = = 1)
                ;

          /* critical section */

        flag[0] = false;

        /* remainder section */

}
```

```
Process P1:
while (true){

        flag[1] = true;
        turn = 0;
        while (flag[0] && turn = = 0)
                ;

          /* critical section */

        flag[1] = false;

        /* remainder section */

}
```



صحت راه کار پترسون

- قابل اثبات است که سه الزام بخش بحرانی رعایت می شوند:

1.انحصار متقابل حفظ می شود:

- فرآیند `Pi` فقط در صورتی وارد بخش بحرانی می شود که :

`flag[j] = false` یا اینکه `turn = i` یا

2.الزام پیشرفت برآورده می شود.

3.الزام انتظار محدود رعایت می شود.



سمافور(راهنما):

- سِمافور ابزاری برای همگام سازی است که روش های پیچیده تری (نسبت به قفل های انحصار متقابل) را برای فرآیندها برای همگام سازی فعالیت هایشان ارائه می دهد.
- سِمافور : S یک متغیر عدد صحیح است.
- فقط از طریق دو عملیات غیرقابل تقسیم (اتمی) قابل دسترسی است.
- دستور wait() یا P / دستور signal() یا V
- تعریف عملیات انتظار

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

تعریف عملیات سیگنال

```
signal(S) {
    S++;
}
```



انواع سمافور

- سِمافور شمارش :مقدار عدد صحیح می تواند روی یک دامنه بدون محدودیت قرار گیرد.
- سِمافور دودویی :مقدار عدد صحیح می تواند تنها بین 0 و ۱ قرار گیرد.
- (Mutex Lock) که مشابه قفل انحصار متقابل است

- ✓ با استفاده از سِمافورها می توانیم مشکالت مختلف همگام سازی را حل کنیم.



مثال کاربرد سمافور

- راهکار برای مسئله بخش بحرانی
  - یک سمافور mutex با مقدار اولیه ۱

```
wait(mutex);
    CS
signal(mutex);
```

- فرآیندهای P1 و P2 را در نظر بگیرید که دارای دو دستورالعمل S1 و S2 هستند و الزام این است که S1 قبل از S2 اتفاق بیفتد.
- یک سِمافور (synch) ایجاد کنید که مقدار اولیه آن 0 است.

```
P1:
    S1;
    signal(synch);
P2:
    wait(synch);
    S2;
```



- ناظر (مانیتور)

- مانیتور ساختاری از زبان برنامه سازی است که همان کار سمافور را انجام می دهد و کنترل آن هم ساده تر است. سمافور اغلب توسط سیستم عامل پشتیبانی می شود و می تواند توسط زبان برنامه سازی نیز پشتیبانی شود، اما مانیتور بایدفقط توسط زبان برنامه سازی پشتیبانی شود. ساختار مانیتور در زبانهای برنامه سازی به صورت برنامه کتابخانه ای پیاده سازی شده است. این به افراد اجازه می دهد تا قفلهای مانیتور را روی هر شیئی بگذارند. مانیتور مولفه ای نرم افزاری، مشتمل بر یک یا چند رویه، دنباله ای از مقدارگذاری های اولیه و داده های محلی است.

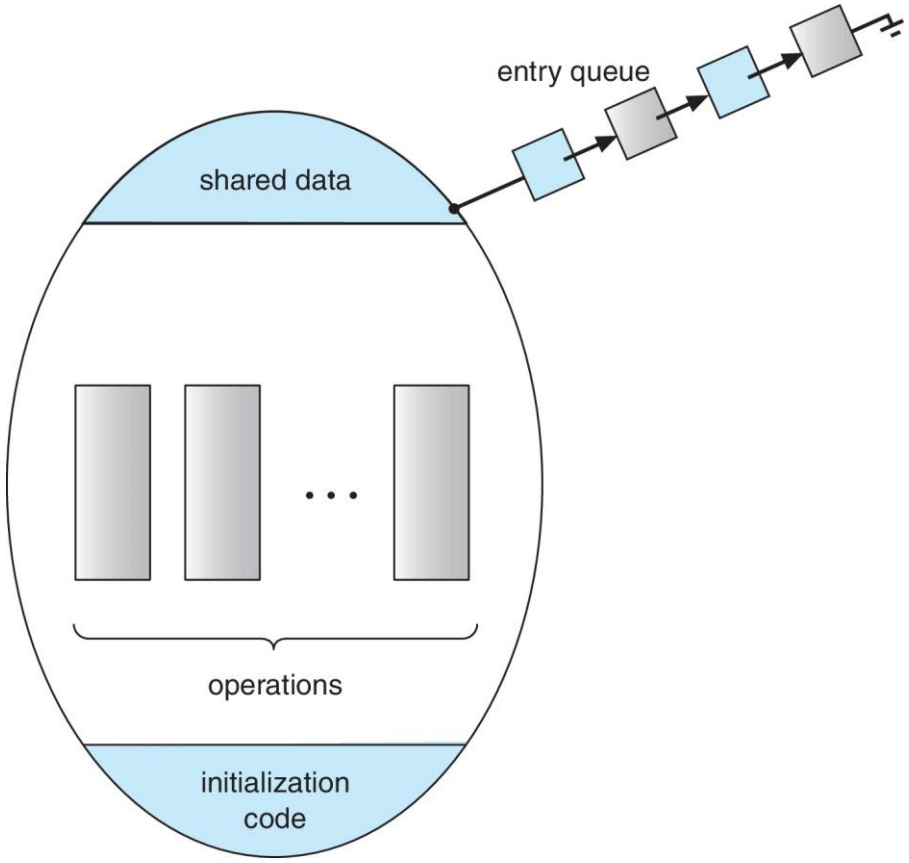


مانیتور

- عبارت monitor تضمین می کند که تنها یک فرآیند در یک زمان درون مانیتور فعال باشد
  - ساختار مانیتور (monitor construct)برای مدل سازی برخی از طرح های همگام سازی به قدر کافی قدرتمند نیست .
  - برای این منظور، ما باید مکانیزم های همگام سازی اضافی تعریف کنیم. این مکانیزم ها توسط ساختار شرط (condition construct)ارائه می شوند.
  - برنامه نویسی که نیاز به نوشتن یک طرح همگام سازی سفارشی دارد، می تواند یک یا چند متغیر از نوع شرط (condition)تعریف کند.



Schematic view of a Monitor



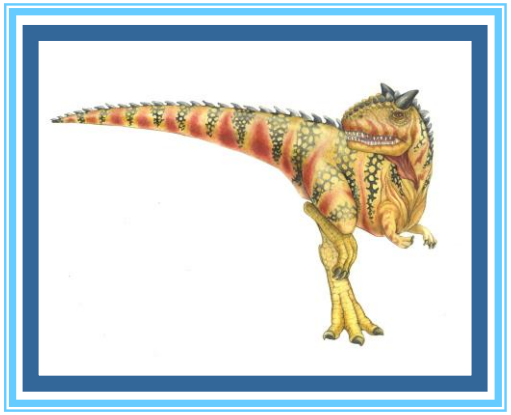


تبادل پیام

- وقتی فرایندها با یکدیگر محاوره می کنند، نیازهای همگام سازی و ارتباط باید تامین شود. فرایندها نیاز به همگام سازی دارند تا انحصار متقابل اعمال گردد. ممکن است فرایندهایی که با یکدیگر همکاری می کنند نیاز به تبادل اطلاعات داشته باشند. یک رویکرد در فراهم کردن این دو عمل، تبادل پیام است. عمل واقعی تبادل پیام به شکل دو دستور `send` و `recive` ارائه می شود.



End of Chapter 6



Silberschatz, Galvin and Gagne ©2013